UTN FACULTAD REGIONAL MENDOZA
UNIVERSIDAD TECNOLÓGICA NACIONAL

Trabajo Practico N° 3 – Programación 4

useEffect + persistencia + componentización + lógica derivada

useEffect

useEffect es un hook que permite ejecutar código automáticamente cuando ocurre algo en el componente, es decir "Sirve para hacer cosas cuando el componente se muestra o cuando cambian datos."

useEffect es como decirle a React:
"cuando pase esto… ejecutá este código"

Es como un **evento automático**

- Cuando abre la pantalla → ejecuta algo
- Cuando cambia un valor → ejecuta algo

Gestión de Participantes Persistente y Modular

Objetivos

El alumno deberá aprender:

- Uso de useEffect
- Persistencia con **LocalStorage**
- Separación en **componentes**
- Manejo de **estado derivado (filtros)**
- Reutilización de código

Evolución respecto al TP1

TP1:

- Todo en un solo componente
- Solo useState

TP2:

- Componentes separados
- **useEffect**
- Datos persistentes
- Lógica más organizada

Estructura requerida

Deberá reestructurar el código fuente de tal manera de respetar la siguiente estructura de archivos.

```
src/
 ├── components/
 │    ├── Formulario.tsx
 │    ├── Filtros.tsx
 │    └── ParticipanteCard.tsx
 ├── models/
 │    └── Participante.ts
 ├── App.tsx
 ├── main.tsx
 └── index.css
```

Requerimientos

El sistema debe:

- Guardar automáticamente los participantes
- Recuperarlos al cargar la página
- Crear componentes para los elementos de la interfaz de usuario

Componentización

Deberá separar las interfaces en los siguientes componentes:

Formulario

- Alta de participante

Sección de Filtros

- Buscar por nombre
- Filtrar por nivel
- Filtrar por modalidad

Tarjeta Participante

- Mostrar datos
- Botón eliminar

Aplicar Props

Props son datos que un componente padre le pasa a un componente hijo.

“Las props permiten que un componente sea reutilizable.”

Cada componente debe recibir datos por **props**

Ejemplo:

```
<Formulario onAgregar={...} />
<Filtros filtros={...} setFiltros={...} />
<ParticipanteCard participante={p} />
```

Filtros combinados

Debe implementar:

- Búsqueda por nombre
- Filtro por nivel
- Filtro por modalidad

Todo junto (AND lógico)

Filtros combinados

El alumno debe implementar:

- Búsqueda por nombre
- Filtro por nivel
- Filtro por modalidad

Todo junto (AND lógico)

Uso de la clase Participante

Debe mantenerse:

new Participante(...)

Funcionalidades Extra

Botón “Limpiar filtros”

Botón “Resetear datos” -> localStorage.removeItem("participantes");

Mensaje vacío

"No hay participantes"

Contador dinámico

Mostrando 3 de 10 participantes